LECTURE NOTES

Linear Predictors

AI512 — Introduction to Machine Learning

Simon Holm

2026-09-04

SDU

DEPARTMENT OF MATHEMATICS
AND COMPUTER SCIENCE

Contents

1 Least Squares Regression 3
2 Metric Spaces 3
3 Regularized Least Squares 4
4 Ridge regression 5
5 Z-score normalization 5
6 Lasso regression 6
6.1 From 02_Linear_Predictors.ipynb 7

1 Least Squares Regression

Assume a dataset Data obviously has a linear correlation. So we can denote the hypothesis space more specifically

$$\mathcal{H} := \{h : h(x) = w^T x, w \in \mathbb{R}^k\}$$

Where $w = (w_0, w_1)$

Assume dataset $S = \{(x_i, y_i) : i \in [m]\}$, then squared-error loss for h is:

$$L_S(w) := \frac{1}{m} \sum_{i \in [m]} (w^\top x_i - y_i)^2$$

Then $w_S := \arg\min_w L_S(w)$ to find a good vector for the hypothesis w

$$\begin{aligned} \nabla_w L_S(w) &= \frac{2}{m} \sum_{i=1}^{m} x_i (x_i^\top w - y_i) \\ &= 0 \end{aligned}$$

$$\Rightarrow \sum_{i=1}^{m} x_i x_i^\top w_S = \sum_{i=1}^{m} x_i y_i$$

$$\Rightarrow w_S = \left(\sum_{i=1}^{m} x_i x_i^\top \right)^{-1} \sum_{i=1}^{m} x_i y_i$$

This is often refered to as least squares regression, and w as the least squares solution.

We can denote $z_i := (x_i, 1)$, where $Z = \begin{pmatrix} z_i \\ \vdots \\ z_n \end{pmatrix}$ to show that:

$$w_S := (Z^\top Z)^{-1} Z^T y.$$

(this is an example of the learned hypothesis h with w)

2 Metric Spaces

We would like the vector spaces (such as feature spaces, parameters spaces etc.) to have some plausible properties such as.

- **strict:** $\forall a, a' \in X, a \neq a' \Rightarrow dist(a, a') > 0$,
- **reflexive:** $\forall a \in X, \Rightarrow dist(a, a) = 0$,
- **symmetric:** $\forall a, a' \in X, \Rightarrow dist(a, a') = dist(a', a)$.
- **triangle inequality**: $\forall a, b, c \in X, dist(a, c) \leq dist(a, b) + dist(b, c)$.

Where $\text{dist}(a, b)$ is a function that measures distance between two points.

These requirements are nice to have since they make the distance function $\text{dist}(a, b)$ behave like a **real notion of distance**.

Using this we can define the **norm** of a vector aka. The length For $p > 0$ the L_p norm is defined as follows:

$$| \mid u \mid |_p = \Big(\sum_{j=1}^{k} | \ u_j|^p\Big)^{1/p}$$

If $p = 2$, we get the well-known **Euclidean norm**.

If $p = 1$, we get the **Manhattan norm**.

As $p \to \infty$, we get the **Maximum norm** (L_∞), i.e. $| \mid u \mid |_\infty := \max\{| \ u_1|, ..., | \ u_d|\}$.

We can use this norm to derive many nice metric spaces

$\text{dist}(a, b) = \|a - b\|_p$.

(this example shows the behavior of the distances w.r.t p)

3 Regularized Least Squares

As we saw in 3. Polynomial curve fitting, $L_S(h_S) = 0$ by memorizing training data can be achieved but will result in overfitting

Assume a hypothesis space H of finite size $|H|$ and a loss function $L_s(h)$ which fits the data as well as possible and constraining model complexity.

$$h_S := \arg\min_{h \in H} L_S(h) + \lambda \mid H \mid$$

With λ as a **regularization coefficient** and $|H|$ the **regularizer** this is called **Structured Risk Minimization**, one of the biggest achievements of machine learning research in the pre-deep-learning era.

Let us apply this to the least squares problem.

$$h_S := \arg\min_w \frac{1}{m} \sum_{i \in [m]} (w^\top x_i - y_i)^2$$

As we know, when m become very large, the weighs will also become large, and thus result in overfitting. To counter this we force the model to keep weights small with the constraint

$$h_S := \arg\min_w \frac{1}{m} \sum_{i \in [m]} (w^\top x_i - y_i)^2, \qquad | \mid w \mid |_p \leq \eta$$

This constrained optimization problem can be expressed equivalently as:

$$h_S := \arg\min_w \max_\lambda \frac{1}{m} \sum_{i \in [m]} (w^\top x_i - y_i)^2 + \lambda(|\ |\ w\ |\ |_p^p - \eta)$$

where $\lambda \geq 0$.

By choosing a large λ and dropping the inner max problem, we can find a reasonable approximation referred to as **regularized least squares**:

$$h_S := \arg\min_w \frac{1}{m} \sum_{i \in [m]} (w^\top x_i - y_i)^2 + \lambda\ |\ |\ w\ |\ |_p^p$$

We can solve for w with $\nabla L_S(h_S) = 0$ to find a good w for the hypothesis

4 Ridge regression

Take a special case of regularized least squares, where $p = 2$ (example of solving for an opt. w)

$$h_S := \arg\min_w \frac{1}{m} \sum_{i \in [m]} (w^\top x_i - y_i)^2 + \lambda\ |\ |\ w\ |\ |_2^2$$

We can rewrite the loss of this optimization in vector form as.

$$L_S(w) := \frac{1}{m}(Zw - y)^2 + \lambda w^\top w$$

Let us find the optimal weights that minimize the loss by setting its gradient to zero once again: (skipped because I can't be bothered smh)

$$L_S(w) = \frac{1}{m} w^\top Z^\top Z w - \frac{1}{m} 2 w^\top Z^\top y + \lambda w^\top w$$

$$\Rightarrow w_S = \left(\frac{1}{m} Z^\top Z + \lambda I\right)^{-1} Z^\top y.$$

This again is known as **ridge regression**.

Note: finding a lambda which serves an optimal penalty, is normally found with **k-fold Cross Validation**

5 Z-score normalization

A model outputs the following:

Means: [−0.00031786 0.00045392 −0.00103062 −0.00226024 −0.00069973 −0.00092399 −0.00058225 0.00039851 0.00066739 −0.00087822] Variances: [0.00222035 0.00226498 0.00217988 0.00219522 0.00233499 0.00234279 0.00229155 0.00229439 0.0023026 0.00222032]

Clearly these means vary a lot in scales (-0.0023 is almost 10x larger than −0.0003)

To avoid unintentionally prioritizing the features with larger numbers we can normalize (using **Z-score normalization**)

Assume that some data is normal distributed with some mean μ and variance σ^2. That is, each dimension x_d of a D —dimensional observation x comes from a sampling process as below:

$$\epsilon \sim N(0,1), \quad x_d = \mu_d + \sigma_d \epsilon$$

We can then

$$x' = \frac{x_d - \mu_d}{\sigma_d} = \epsilon$$

And thus we can center and scale based on the sample mean μ and standard deviation σ.

6 Lasso regression

Much like ridge regression where

$$h_S := \arg\min_w \frac{1}{m} \sum_{i \in [m]} (w^\top x_i - y_i)^2 + \lambda \mid \mid w \mid \mid_2^2$$

Where the penalty term is $\lambda \|w\|_2^2$

In Lasso regression, the penalty term is simply $\lambda |w|$

This way Lasso "selects" features by letting only the most predictive ones survive while others are pushed exactly to zero. This is unlike ridge regression, where terms can only approach 0.

So with Lasso regression

- If a feature is **strongly correlated with the output**, it tends to survive (keep a nonzero coefficient).
- If a feature is **weakly correlated** or **highly redundant compared with other features**, Lasso often sets it to zero.

This however results in a problem since,

$$f'(x) = \begin{cases} 1, & \text{if } x > 0 \\ [2mm] - 1, & \text{if } x < 0 \\ [1mm] \text{ undefined}, & \text{if } x = 0 \end{cases}$$

Because of this we cannot solve for w using $\nabla L_S(h_S) = 0$

However, we can use gradient descent in the direction $-\nabla L_S(h_S)$ (the direction opposite of the greatest uphill) to approach an optimal w where $\nabla L_S(h_S)$ is as small as possible.

$$w_{t+1} := w_t - \alpha \nabla_w L_S(w) \mid_{w:=w_t}$$

This approach is called **gradient descent**. It is in use with nearly all modern machine learning approaches. The coefficient $\alpha > 0$ is called a **learning rate**.

Gradient descent requires repetitive evaluation of the gradient of the loss with respect to the parameters at every iteration: $L_S(w) \mid_{w:=w_t}$. Hence, its implementation on the computer requires an analytical calculation of this gradient. This may be time consuming for complex loss functions. Deep learning libraries such as PyTorch and TensorFlow allow us to automate this process. See an example PyTorch implementation of Lasso regression below.

6.1 From 02_Linear_Predictors.ipynb

Python

```python
import torch
as th
import torch.nn as nn
import torch.nn.functional as F
import torch.optim

class LassoRegression(nn.Module):
  def __init__(self, n_dims, lambda_coef=1):
  super(LassoRegression, self).__init__()
  self.lambda_coef = lambda_coef
  self.emp_risk = nn.MSELoss()
  self.weight = nn.Parameter(th.randn((n_dims,1)))
  self.bias = nn.Parameter(th.randn((1)))

  def predict(self, input):
    return input @ self.weight + self.bias

  def learn(self, inputs, labels, num_steps=1):
    # The in-built Stochastic Gradient Descent optimizer
    # The argument "lr" sets the learning rate
    optimizer = torch.optim.SGD(self.parameters(), lr=0.01)

    for ii in range(num_steps):
      # Predict with the current weight values
```

```
        # This step is called a "forward pass"

        predictions = self.predict(inputs)
        loss = ((predictions - labels)**2).mean() \
            + self.weight.abs().sum()*self.lambda_coef

        # Clear the gradient values remaining from
        # the previous iteration
        optimizer.zero_grad()

        # Compute the new gradient values
        # This step is called the "backward pass"
        loss.backward()

        # Take the gradient descent step
        optimizer.step()

# Convert data into the Torch format
X_train = torch.tensor(X_train).float()
X_test = torch.tensor(X_test).float()
y_train = torch.tensor(y_train).float().reshape(-1,1)
y_test = torch.tensor(y_test).float().reshape(-1,1)

# z-score normalization
m = th.mean(X_train,axis=0)
std = th.std(X_train,axis=0)
X_train = (X_train-m)/std
X_test = (X_test-m)/std
# Train our model
model_lasso = LassoRegression(n_dims=X_train.shape[1], lambda_coef=1)

# Number of gradient descent iterations
num_iterations = 250

# Collect the train and test errors here.
train_errors = np.zeros(num_iterations)
test_errors = np.zeros(num_iterations)

```

```
for ii in range(num_iterations):
  model_lasso.learn(X_train, y_train)
  predictions = model_lasso.predict(X_train)
  train_error = ((predictions - y_train)**2).mean().sqrt()
  train_errors[ii] = train_error.detach().numpy()

  # Test our model
  predictions = model_lasso.predict(X_test)
  test_error = ((predictions - y_test)**2).mean().sqrt()
  test_errors[ii] = test_error.detach().numpy()

# Plot the learning curve
plt.plot(np.arange(num_iterations),train_errors,'b-', label="Train RMSE")
plt.plot(np.arange(num_iterations),test_errors,'r-', label="Test RMSE")
plt.xlabel("Iteration")
plt.ylabel("RMSE")
plt.legend(loc="upper right")
plt.show()
```

(plotted graph from lecture)